

UDESC

Departamento de Ciência da Computação

Teoria dos Grafos: Tarefa 2

**Implementação do algoritmo de Dijkstra para distância
entre palavras a partir de uma Lista de Adjacências**

Alunos: Heitor Linhares Caetano e Deivid Veras Lourenço

Professor: Gilmário Barbosa dos Santos

Junho

2026

Sumário

1	Objetivo do Trabalho	1
2	Solução fornecida	1
2.1	Construção do grafo	1
2.1.1	Determinação de arestas	2
2.2	Determinação das palavras com mais e menos conexões	2
2.3	Quantidade de laços e arestas múltiplas	2
2.4	Algoritmo de Dijkstra	2
3	Sobre o projeto	3
3.1	Ambiente de desenvolvimento	3
3.2	Bibliotecas utilizadas	3
3.3	Instruções de compilação	3
4	Análise dos Resultados	5
4.1	Resultados obtidos	6
5	Considerações Finais	8

1 Objetivo do Trabalho

A atividade constituiu em implementar um grafo não direcionado ponderado onde os vértices são palavras de 4 letras cada, e as arestas conectam palavras que diferem por apenas uma letra.

Além disso, outros requisitos eram indicar a palavra com mais e com menos conexões do grafo, além de implementar o algoritmo de Dijkstra para encontrar o menor caminho possível entre duas palavras arbitrárias.

2 Solução fornecida

Utilizamos o trabalho anterior como base para este. Isso significa que a implementação da representação computacional se manteve a mesma (sendo uma lista de adjacências), com a única mudança sendo a implementação de pesos entre as arestas.

Como a distância entre as palavras é, por definição do grafo, sempre de valor um, os pesos das arestas também serão sempre um. Isso cria um grafo uniformemente ponderado, em que é possível realizar a busca de caminho mínimo através de uma busca por largura (BFS). Mesmo com essa informação, mantivemos a implementação com o algoritmo de Dijkstra para atender exatamente o que foi pedido.

2.1 Construção do grafo

A construção do grafo foi uma das tarefas mais trabalhosas do projeto. Como agora o identificador de cada vértice é uma palavra, e não um número, não é possível estabelecer uma relação direta e natural entre o nome do vértice e seu índice.

Para resolver esse problema, decidimos criar um tipo de dado composto (*struct*) chamado GrafoPalavras. Ele tem como campos uma Lista de adjacências (implementada previamente), um vetor de *strings* que faz a associação índice $\rightarrow$ palavra, um valor inteiro que representa a quantidade de palavras (vértices) em uso e, por fim, uma tabela *Hash* que faz a associação palavra $\rightarrow$ índice.

O uso de uma tabela Hash faz com que buscas por índices de palavras sejam de complexidade constante ($O(1)$), sendo muito mais eficientes do que as possíveis alternativas (como atravessar por todo o vetor, que seria $O(n)$).

2.1.1 Determinação de arestas

Para determinar quando uma palavra deve ser adjacente a outra, é preciso comparar caractere por caractere, criando a aresta somente quando as duas divergem por apenas uma letra.

Aqui já encontramos o primeiro dilema do projeto. A diferença entre palavras deve ser *case-sensitive*? Ou seja, devemos tratar letras em caixa alta e caixa baixa como diferentes, fazendo com que a palavra IMPA não seja associada a palavra impo?

Decidimos que não. Faz mais sentido tratarmos letras maiúsculas e minúsculas como iguais, pois muitas vezes a capitalização não é algo essencial das palavras, mas apenas um fator acidental (como a maiusculização da letra inicial da primeira palavra de um parágrafo, por exemplo).

Mesmo assim, optamos por deixar a sensibilidade à caixa alta como uma opção ao usuário, disponível como parâmetro em tempo de compilação. Instruções disponíveis na próxima seção (3.3).

2.2 Determinação das palavras com mais e menos conexões

Essa parte foi mais direta. Estendemos as funções do trabalho anterior que retornavam o maior e menor grau em uma função que retorna também os índices de todos os vértices com aquele grau, através de um tipo de dado composto especializado. A partir disso, com o vetor *palavras* que faz parte do tipo GrafoPalavras, é possível encontrar as palavras associadas aos números.

2.3 Quantidade de laços e arestas múltiplas

Aqui não foi preciso alterar a implementação, pois o código do trabalho anterior serve perfeitamente neste. Para mais informações sobre essa parte, veja a seção de mesmo nome no relatório anterior, disponível [aqui](#).

2.4 Algoritmo de Dijkstra

3 Sobre o projeto

O projeto foi administrado em um [repositório](#) no GitHub. Como indicado anteriormente, ele inicialmente começou como um *fork* do trabalho anterior, lentamente se tornando uma coisa própria.

Para a organização das tarefas, criamos um arquivo `TODO.md` que lista o que deve ser feito e o que já foi realizado.

3.1 Ambiente de desenvolvimento

Essa é outra parte que não mudou muito. Deivid usou o sistema operacional Windows e o editor de código [Visual Studio Code](#), enquanto Heitor usou uma distribuição GNU/Linux e o editor de texto [Helix](#).

3.2 Bibliotecas utilizadas

Além de bibliotecas padrão do C como `stdlib`, `stdio` e `string.h`, utilizamos uma biblioteca externa chamada `libfort`. Essa é uma biblioteca relativamente pequena, que nos dá um jeito prático de criar tabelas formatadas usando apenas texto. Ela foi aplicada apenas no arquivo `main.c`, e não teve impacto na implementação do que foi pedido em si.

O código da biblioteca está hospedado em:

<https://github.com/seleznevae/libfort/>

3.3 Instruções de compilação

O comando para compilar em ambientes GNU/Linux é:

```
gcc ./vendor/fort.c ls_adj.c hash.c arquivo.c main.  
c -O2 -o grafo
```

É disponibilizado um arquivo de script chamado `compilar.sh` que realiza essa instrução automaticamente. Ele está localizado no diretório `src/`.

Para iniciar o programa, simplesmente execute:

```
./grafo
```

Para utilizar o algoritmo de busca em largura (BFS) em vez da busca em profundidade na determinação dos componentes conexos, basta adicionar o parâmetro `-DBFS` ao comando de compilação:

```
gcc ./vendor/fort.c ls_adj.c hash.c arquivo.c main.  
c -O2 -o grafo -DBFS
```

Já para tratar letras maiúsculas e minúsculas como símbolos diferentes, tornando assim o grafo *case-sensitive*, adicione o parâmetro **-DCASE_SENSITIVE**:

```
gcc ./vendor/fort.c ls_adj.c hash.c arquivo.c main.  
c -O2 -o grafo -DCASE_SENSITIVE
```

4 Análise dos Resultados

Antes de partir pros resultados, podemos tentar visualizar o grafo através de um programa dedicado a isso. Continuando o trabalho anterior, utilizamos o software [GraphViz](#) para isso:

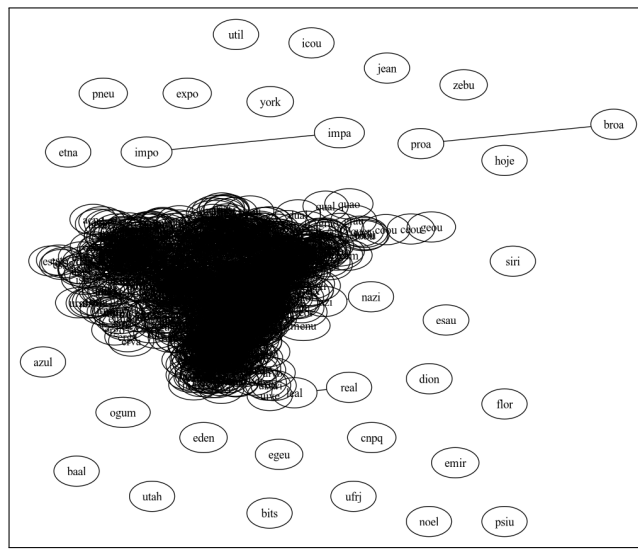


Figura 1: Visualização do grafo através do software GraphViz.

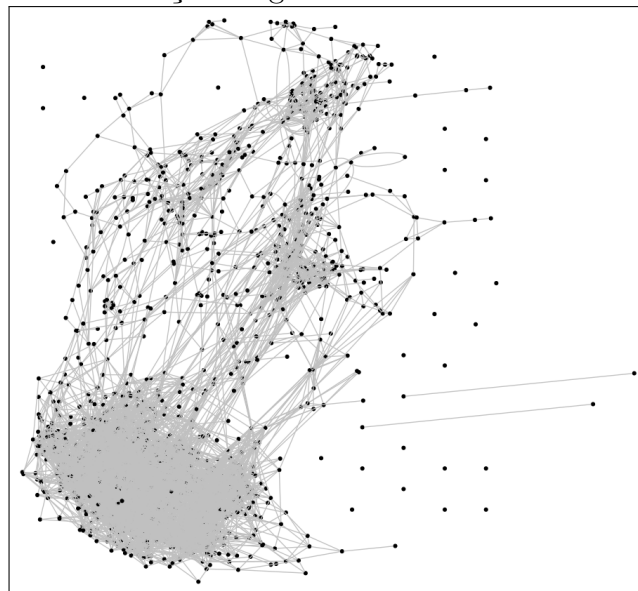


Figura 2: Visualização com os vértices transformados em pontos.

Pelas imagens, é possível observar que há um grande componente conexo

aresta para cada duas palavras que diferem em apenas um caractere, é impossível a existência tanto de laços quanto de arestas múltiplas.

Já na análise de componentes conexos, foram identificados 28 componentes, sendo um de tamanho 1575, dois de tamanho 2 e o restante de tamanho 1 (vértices isolados/nulos). Isso confirma nossa análise prévia, e mostra uma enorme desigualdade no grafo.

Mesmo sendo um "*overkill*", também criamos um histograma para a distribuição de componentes conexos, usando a ajuda do software [gnuplot](#):

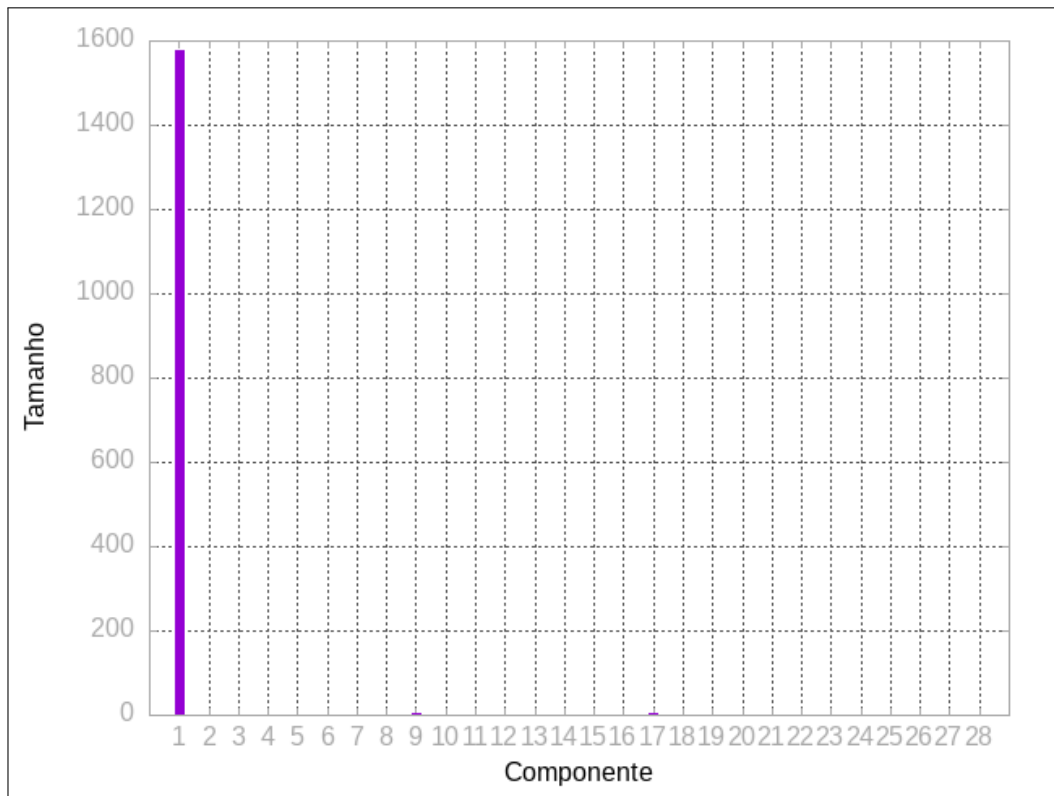


Figura 4: Histograma.

5 Considerações Finais